Two failure modes: optimization, generalization

AI Methods for Science – Part I – Week 2, Monday

Yohai Bar-Sinai

October 22, 2026

Where we are

1. A flexible function f_θ
still the MLP
from the lab

2. A cost function $L(\theta)$
the loss on new data

3. An optimizer
why it stalls, what fixes it

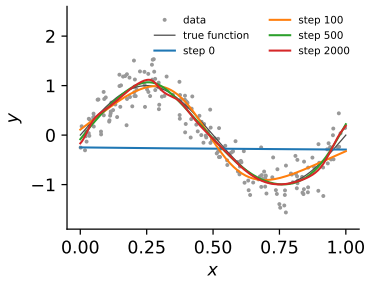
Today, in this order:

1. your reports, E0–E8, next to our runs (20 min)
2. two failure modes in eight frames (35 min)
3. lab: E9–E11, then Part C, E12–E15, to the end of the sheet (55 min)
4. your curves next to ours (10 min)

Failure	Symptom
Optimization	training loss stays high
Approximation	the family cannot represent the answer (Thursday)
Generalization	training loss low, held-out loss high

Your Part A and E1–E4, next to ours

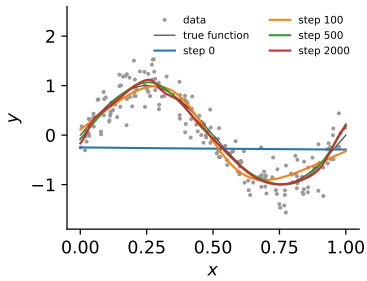
Part A: fit a function, from our runs



`demos/first_task.py`; MLP 1–64–64–1, tanh, Adam, full batch, 200 points, $\sigma = 0.25$.

run	train MSE	test MSE
noise floor on this sample ($\sigma^2 = 0.0625$)	0.070	0.055
Adam 10^{-2}, 2000 steps	0.066	0.058
SGD 0.05	0.087	0.068
momentum 0.9, 10^{-2}	0.077	0.060
width 1	0.151	0.121
5000 steps	0.050	0.077
$n = 15$, 5000 steps	0.000	0.289

Part A: fit a function, from our runs

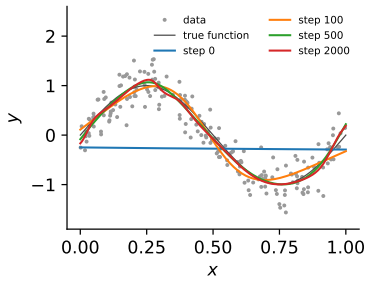


`demos/first_task.py`; MLP 1–64–64–1, tanh, Adam, full batch, 200 points, $\sigma = 0.25$.

run	train MSE	test MSE
noise floor on this sample ($\sigma^2 = 0.0625$)	0.070	0.055
Adam 10^{-2}, 2000 steps	0.066	0.058
SGD 0.05	0.087	0.068
momentum 0.9, 10^{-2}	0.077	0.060
width 1	0.151	0.121
5000 steps	0.050	0.077
$n = 15$, 5000 steps	0.000	0.289

- the floor is the MSE of the *true* function on each sample; train MSE below it is fitting the noise

Part A: fit a function, from our runs

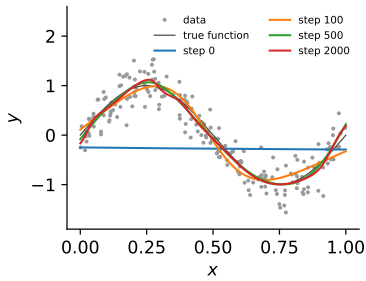


`demos/first_task.py`; MLP 1–64–64–1, tanh, Adam, full batch, 200 points, $\sigma = 0.25$.

run	train MSE	test MSE
noise floor on this sample ($\sigma^2 = 0.0625$)	0.070	0.055
Adam 10^{-2}, 2000 steps	0.066	0.058
SGD 0.05	0.087	0.068
momentum 0.9, 10^{-2}	0.077	0.060
width 1	0.151	0.121
5000 steps	0.050	0.077
$n = 15$, 5000 steps	0.000	0.289

- the floor is the MSE of the *true* function on each sample; train MSE below it is fitting the noise
- width 1 underfits; 5000 steps: train below the floor, test up; 15 points: train 0, test $5\times$ the floor. The three failure modes, on 200 numbers

Part A: fit a function, from our runs



`demos/first_task.py`; MLP 1–64–64–1, tanh, Adam, full batch, 200 points, $\sigma = 0.25$.

run	train MSE	test MSE
noise floor on this sample ($\sigma^2 = 0.0625$)	0.070	0.055
Adam 10^{-2}, 2000 steps	0.066	0.058
SGD 0.05	0.087	0.068
momentum 0.9, 10^{-2}	0.077	0.060
width 1	0.151	0.121
5000 steps	0.050	0.077
$n = 15$, 5000 steps	0.000	0.289

- the floor is the MSE of the *true* function on each sample; train MSE below it is fitting the noise
- width 1 underfits; 5000 steps: train below the floor, test up; 15 points: train 0, test $5\times$ the floor. The three failure modes, on 200 numbers
- the optimizers differ in speed, not in where they end

Part A: the reference solution

```
x = torch.rand(200, 1) # data
y = torch.sin(2 * torch.pi * x) + 0.25 * torch.randn
     (200, 1)

model = nn.Sequential( # ansatz
    nn.Linear(1, 64), nn.Tanh(),
    nn.Linear(64, 64), nn.Tanh(),
    nn.Linear(64, 1))
loss_fn = nn.MSELoss() # loss
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for step in range(2000): # the loop
    opt.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()

with torch.no_grad(): # the floor
    floor = loss_fn(torch.sin(2 * torch.pi * x), y)
```

Full script with the figures and the spiral: `demos/first_task.py`.

Part A: the reference solution

```
x = torch.rand(200, 1) # data
y = torch.sin(2 * torch.pi * x) + 0.25 * torch.randn
     (200, 1)

model = nn.Sequential( # ansatz
    nn.Linear(1, 64), nn.Tanh(),
    nn.Linear(64, 64), nn.Tanh(),
    nn.Linear(64, 1))
loss_fn = nn.MSELoss() # loss
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for step in range(2000): # the loop
    opt.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()

with torch.no_grad(): # the floor
    floor = loss_fn(torch.sin(2 * torch.pi * x), y)
```

- three boxes: ansatz, loss, optimizer; then the same five lines every time

Full script with the figures and the spiral: `demos/first_task.py`.

Part A: the reference solution

```
x = torch.rand(200, 1) # data
y = torch.sin(2 * torch.pi * x) + 0.25 * torch.randn
     (200, 1)

model = nn.Sequential( # ansatz
    nn.Linear(1, 64), nn.Tanh(),
    nn.Linear(64, 64), nn.Tanh(),
    nn.Linear(64, 1))
loss_fn = nn.MSELoss() # loss
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for step in range(2000): # the loop
    opt.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()

with torch.no_grad(): # the floor
    floor = loss_fn(torch.sin(2 * torch.pi * x), y)
```

- three boxes: ansatz, loss, optimizer; then the same five lines every time
- full batch: all 200 points in every step. Gradient descent, no noise.

Full script with the figures and the spiral: `demos/first_task.py`.

Part A: the reference solution

```
x = torch.rand(200, 1) # data
y = torch.sin(2 * torch.pi * x) + 0.25 * torch.randn
     (200, 1)

model = nn.Sequential( # ansatz
    nn.Linear(1, 64), nn.Tanh(),
    nn.Linear(64, 64), nn.Tanh(),
    nn.Linear(64, 1))
loss_fn = nn.MSELoss() # loss
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for step in range(2000): # the loop
    opt.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()

with torch.no_grad(): # the floor
    floor = loss_fn(torch.sin(2 * torch.pi * x), y)
```

- three boxes: ansatz, loss, optimizer; then the same five lines every time
- full batch: all 200 points in every step. Gradient descent, no noise.
- the floor: the true function on the same sample, so not exactly σ^2

Full script with the figures and the spiral: `demos/first_task.py`.

Part A: the reference solution

```
x = torch.rand(200, 1) # data
y = torch.sin(2 * torch.pi * x) + 0.25 * torch.randn
     (200, 1)

model = nn.Sequential( # ansatz
    nn.Linear(1, 64), nn.Tanh(),
    nn.Linear(64, 64), nn.Tanh(),
    nn.Linear(64, 1))
loss_fn = nn.MSELoss() # loss
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

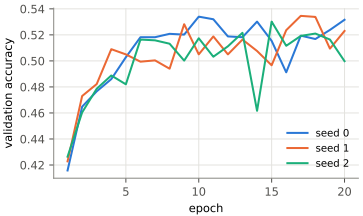
for step in range(2000): # the loop
    opt.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()

with torch.no_grad(): # the floor
    floor = loss_fn(torch.sin(2 * torch.pi * x), y)
```

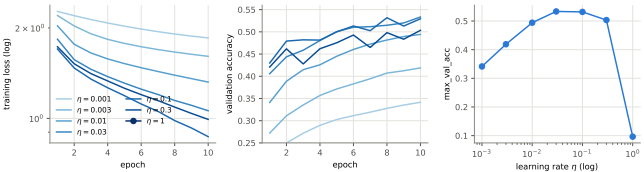
- three boxes: ansatz, loss, optimizer; then the same five lines every time
- full batch: all 200 points in every step. Gradient descent, no noise.
- the floor: the true function on the same sample, so not exactly σ^2
- each variation is one line: `SGD(lr=0.05)`, `momentum=0.9`, `Linear(1, 1)`, `range(5000)`, `rand(15, 1)`

Full script with the figures and the spiral: `demos/first_task.py`.

E1, E2: the noise, and the learning rate

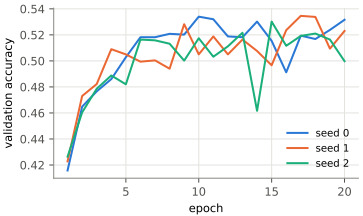


E1: three seeds, 51.6–52.8 at the best epoch. The yardstick: 1–3 points.

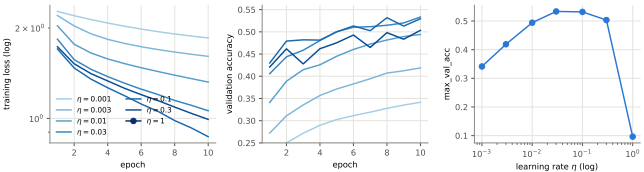


E2: crawl (10^{-3}), work (0.03–0.1), oscillate (0.3: validation jumps 3 points per epoch), diverge (1).

E1, E2: the noise, and the learning rate



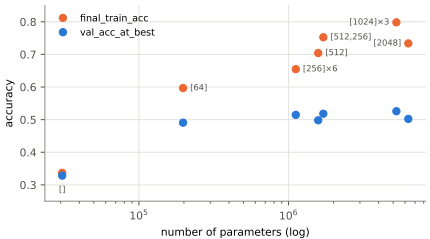
E1: three seeds, 51.6–52.8 at the best epoch. The yardstick: 1–3 points.



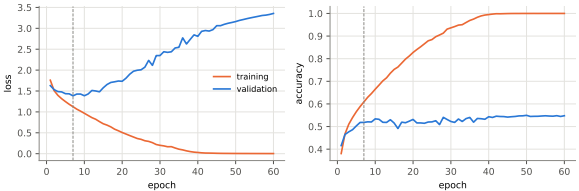
E2: crawl (10^{-3}), work (0.03–0.1), oscillate (0.3: validation jumps 3 points per epoch), diverge (1).

Both are today's: the three regimes are the *condition number*, the jumps and the seed spread are the *noise*.

E3, E4: capacity, and training too long

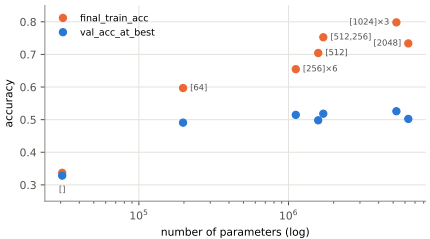


E3: linear 33; from 1M to 6M parameters everything is 50–53. More parameters did not hurt.

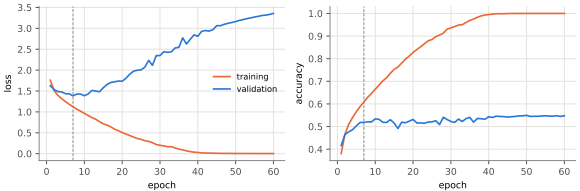


E4: training accuracy 100% by epoch 52; validation loss lowest at epoch 7 then $\times 2.4$; validation accuracy still creeps to 55.

E3, E4: capacity, and training too long



E3: linear 33; from 1M to 6M parameters everything is 50–53. More parameters did not hurt.

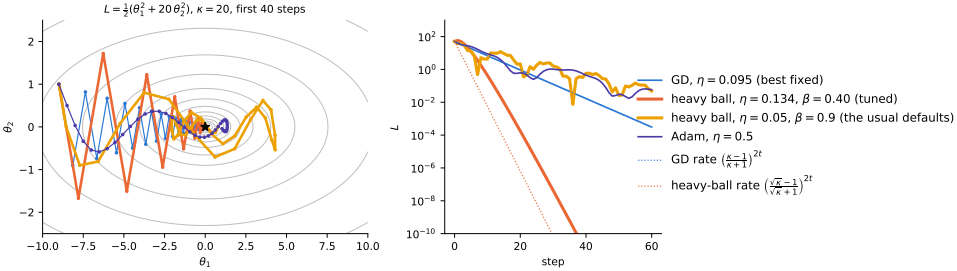


E4: training accuracy 100% by epoch 52; validation loss lowest at epoch 7 then $\times 2.4$; validation accuracy still creeps to 55.

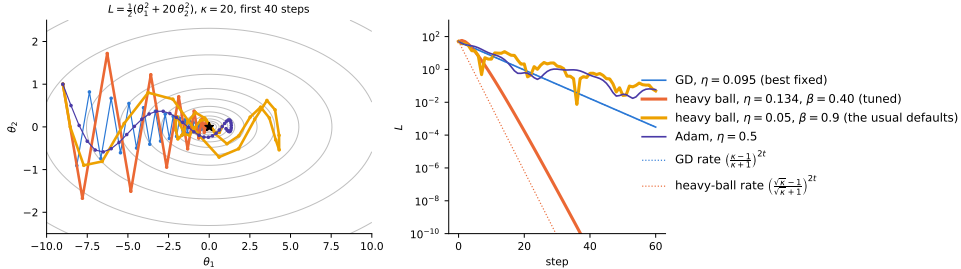
Both are the generalization failure: the network memorizes, and capacity is not the knob that fixes it.

Optimization

Conditioning: why plain GD crawls, and what momentum does

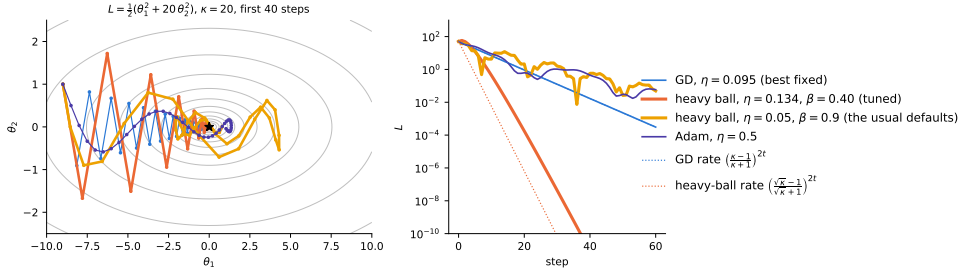


Conditioning: why plain GD crawls, and what momentum does



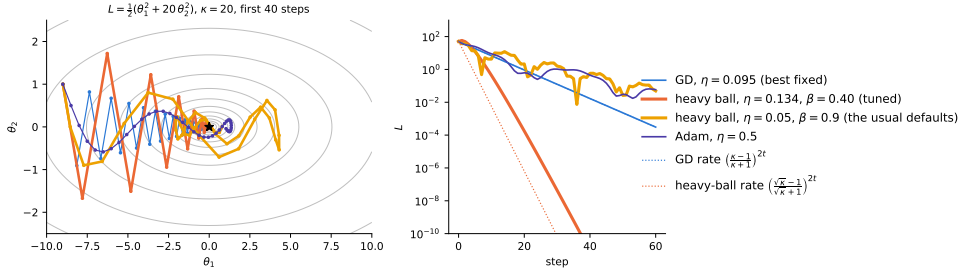
- Near a minimum $L \approx \frac{1}{2}\mathbf{e}^\top H \mathbf{e}$, H the Hessian, eigenvalues λ_i the curvatures. **Condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$: steepest curvature over flattest. The sharpest direction caps η ($\eta < 2/\lambda_{\max}$); the flattest sets the time, $\sim \kappa$ steps. E2's regimes. Raw pixels (E8): the first layer's $\lambda_{\max}$ is $10^4\times$ larger.

Conditioning: why plain GD crawls, and what momentum does



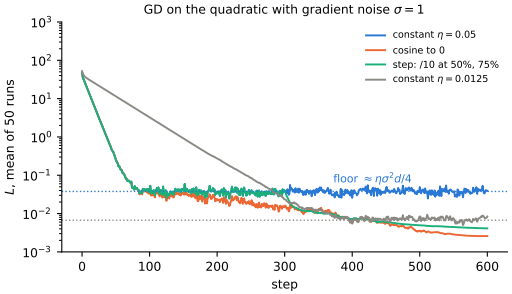
- Near a minimum $L \approx \frac{1}{2}\mathbf{e}^\top H \mathbf{e}$, H the Hessian, eigenvalues λ_i the curvatures. **Condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$: steepest curvature over flattest. The sharpest direction caps η ($\eta < 2/\lambda_{\max}$); the flattest sets the time, $\sim \kappa$ steps. E2's regimes. Raw pixels (E8): the first layer's $\lambda_{\max}$ is $10^4\times$ larger.
- Momentum, $v \leftarrow \beta v + g$: with η, β tuned every mode is damped at the same rate and the time is $\sim \sqrt{\kappa}$ (Polyak 1964). $\kappa = 10^4$: $100\times$ faster. With the usual $\beta = 0.9$: the step is $\eta/(1-\beta) = 10\eta$ along consistent directions, so η is $10\times$ smaller.

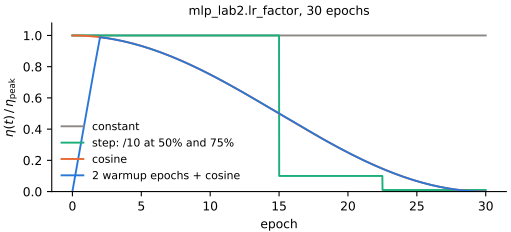
Conditioning: why plain GD crawls, and what momentum does



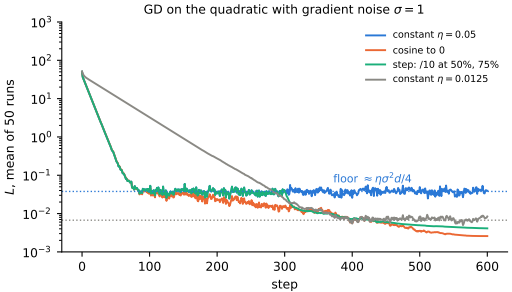
- Near a minimum $L \approx \frac{1}{2}\mathbf{e}^\top H \mathbf{e}$, H the Hessian, eigenvalues λ_i the curvatures. **Condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$: steepest curvature over flattest. The sharpest direction caps η ($\eta < 2/\lambda_{\max}$); the flattest sets the time, $\sim \kappa$ steps. E2's regimes. Raw pixels (E8): the first layer's $\lambda_{\max}$ is $10^4\times$ larger.
- Momentum, $v \leftarrow \beta v + g$: with η, β tuned every mode is damped at the same rate and the time is $\sim \sqrt{\kappa}$ (Polyak 1964). $\kappa = 10^4$: $100\times$ faster. With the usual $\beta = 0.9$: the step is $\eta/(1-\beta) = 10\eta$ along consistent directions, so η is $10\times$ smaller.
- Adam moves every coordinate $\approx \eta$ per step whatever the gradient's size: the conditioning is gone, but it walks around the minimum unless η decays.

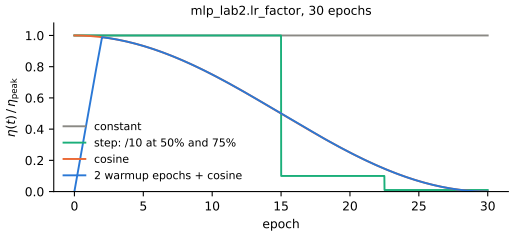
Noise: the floor, and why the learning rate is scheduled





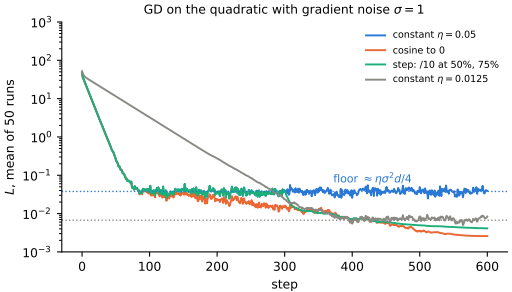
Noise: the floor, and why the learning rate is scheduled

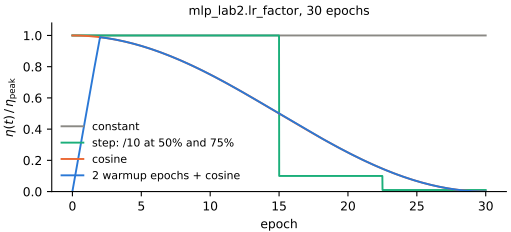




- Minibatch gradients are noisy; SGD at fixed η never converges. It sits on a floor $\propto \eta/B$, independent of the curvature. E2 at 0.3, E1's spread, E5.

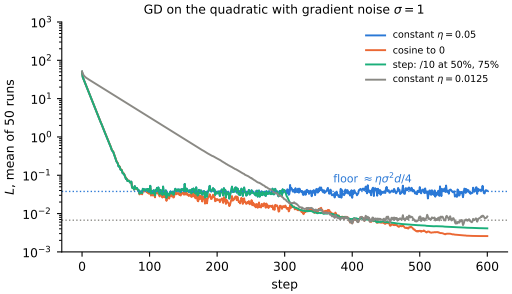
Noise: the floor, and why the learning rate is scheduled

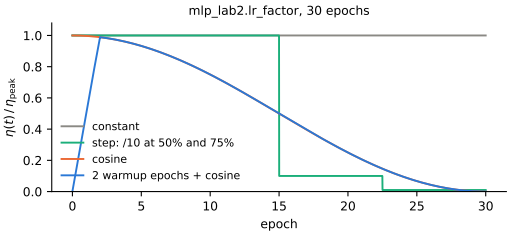




- Minibatch gradients are noisy; SGD at fixed η never converges. It sits on a floor $\propto \eta/B$, independent of the curvature. E2 at 0.3, E1's spread, E5.
- Halve η: half the floor, twice the time. So: decay η over the run. Cosine to zero (`schedule="cosine"`) makes the *last* epoch the best one. Warmup for an unstable start.

Noise: the floor, and why the learning rate is scheduled





- Minibatch gradients are noisy; SGD at fixed η never converges. It sits on a floor $\propto \eta/B$, independent of the curvature. E2 at 0.3, E1's spread, E5.
- Halve η: half the floor, twice the time. So: decay η over the run. Cosine to zero (`schedule="cosine"`) makes the *last* epoch the best one. Warmup for an unstable start.
- Double B and double η: same floor, half the steps (the linear scaling rule, E5's 0.4 at $B = 1024$).

Adam: one learning rate per parameter

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \qquad \theta \leftarrow \theta - \eta \, \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$ (elementwise; session 2)

Adam: one learning rate per parameter

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \qquad \theta \leftarrow \theta - \eta \, \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$ (elementwise; session 2)

- Divides each coordinate's step by its own typical gradient size: a parameter with gradients $10^4\times$ larger gets a step $10^4\times$ smaller.

Adam: one learning rate per parameter

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \qquad \theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$ (elementwise; session 2)

- Divides each coordinate's step by its own typical gradient size: a parameter with gradients $10^4\times$ larger gets a step $10^4\times$ smaller.
- What it does not fix: the noise floor (same η/B, now in units of η), overfitting, correlations between parameters. It still needs a schedule.

Adam: one learning rate per parameter

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \qquad \theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$ (elementwise; session 2)

- Divides each coordinate's step by its own typical gradient size: a parameter with gradients $10^4\times$ larger gets a step $10^4\times$ smaller.
- What it does not fix: the noise floor (same η/B, now in units of η), overfitting, correlations between parameters. It still needs a schedule.
- **AdamW**: weight decay applied to θ directly instead of through $\sqrt{\hat{v}}$ (Loshchilov & Hutter 2019). The default to try first: AdamW, $\eta = 10^{-3}$, $\lambda = 0.01$–0.1, cosine. SGD + momentum 0.9, η from a sweep, for CNNs on images.

Adam: one learning rate per parameter

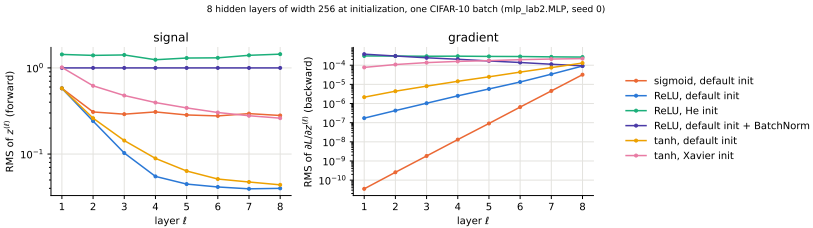
$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \qquad \theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$ (elementwise; session 2)

- Divides each coordinate's step by its own typical gradient size: a parameter with gradients $10^4\times$ larger gets a step $10^4\times$ smaller.
- What it does not fix: the noise floor (same η/B, now in units of η), overfitting, correlations between parameters. It still needs a schedule.
- **AdamW**: weight decay applied to θ directly instead of through $\sqrt{\hat{v}}$ (Loshchilov & Hutter 2019). The default to try first: AdamW, $\eta = 10^{-3}$, $\lambda = 0.01$–0.1, cosine. SGD + momentum 0.9, η from a sweep, for CNNs on images.

Key point

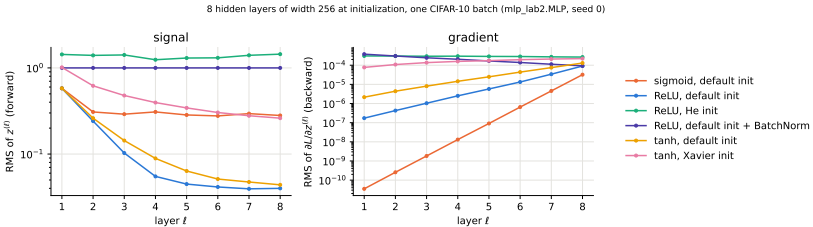
At their best η, every optimizer lands this MLP at the same place (E12, in a minute). The optimizer buys speed and robustness to η, not accuracy.

Depth: the signal through the layers



Plotted: E7's network (8 hidden layers of 256) at initialization, before training, one batch of 256 CIFAR-10 images. Left: RMS of the pre-activations $\mathbf{z}^{(l)}$ per layer, the forward signal. Right: RMS of $\partial L/\partial \mathbf{z}^{(l)}$ per layer, the backward gradient. RMS over the 256 images and the 256 units.

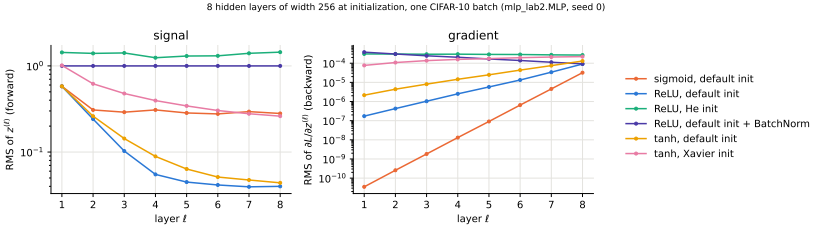
Depth: the signal through the layers



Plotted: E7's network (8 hidden layers of 256) at initialization, before training, one batch of 256 CIFAR-10 images. Left: RMS of the pre-activations $\mathbf{z}^{(l)}$ per layer, the forward signal. Right: RMS of $\partial L/\partial \mathbf{z}^{(l)}$ per layer, the backward gradient. RMS over the 256 images and the 256 units.

- Forward: $\mathbb{E}[z^2] \to n\sigma_W^2\,\mathbb{E}[a^2]$ per layer; backward the same product with ϕ'. One factor per layer, raised to the depth. PyTorch's default init shrinks a ReLU signal by 0.41 per layer: by layer 5 only the biases are left, the output ignores the input, loss $\ln 10$ (E7).

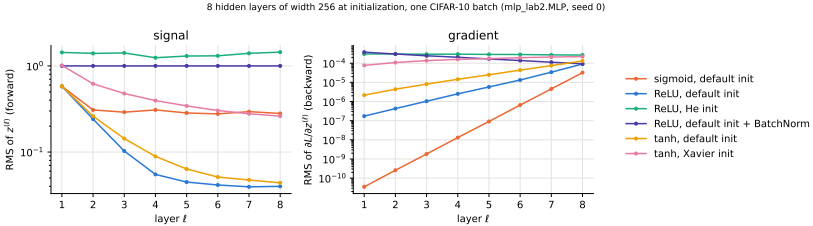
Depth: the signal through the layers



Plotted: E7's network (8 hidden layers of 256) at initialization, before training, one batch of 256 CIFAR-10 images. Left: RMS of the pre-activations $\mathbf{z}^{(l)}$ per layer, the forward signal. Right: RMS of $\partial L/\partial \mathbf{z}^{(l)}$ per layer, the backward gradient. RMS over the 256 images and the 256 units.

- Forward: $\mathbb{E}[z^2] \to n\sigma_W^2\,\mathbb{E}[a^2]$ per layer; backward the same product with ϕ'. One factor per layer, raised to the depth. PyTorch's default init shrinks a ReLU signal by 0.41 per layer: by layer 5 only the biases are left, the output ignores the input, loss $\ln 10$ (E7).
- **He init** ($\sigma_W^2 = 2/n$) keeps the factor at 1 for ReLU (flat green line). Sigmoid: $\phi' \le \frac{1}{4}$ and the offset $\frac{1}{2}$, no init fixes it.

Depth: the signal through the layers

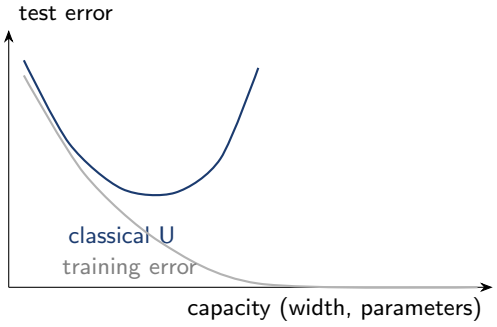


Plotted: E7's network (8 hidden layers of 256) at initialization, before training, one batch of 256 CIFAR-10 images. Left: RMS of the pre-activations $\mathbf{z}^{(l)}$ per layer, the forward signal. Right: RMS of $\partial L/\partial \mathbf{z}^{(l)}$ per layer, the backward gradient. RMS over the 256 images and the 256 units.

- Forward: $\mathbb{E}[z^2] \to n\sigma_W^2\,\mathbb{E}[a^2]$ per layer; backward the same product with ϕ'. One factor per layer, raised to the depth. PyTorch's default init shrinks a ReLU signal by 0.41 per layer: by layer 5 only the biases are left, the output ignores the input, loss $\ln 10$ (E7).
- **He init** ($\sigma_W^2 = 2/n$) keeps the factor at 1 for ReLU (flat green line). Sigmoid: $\phi' \le \frac{1}{4}$ and the offset $\frac{1}{2}$, no init fixes it.
- **BatchNorm**: standardize every layer's pre-activations over the batch, then learn a scale and shift. $O(1)$ by construction. Next: skip connections, the third fix.

Regularization

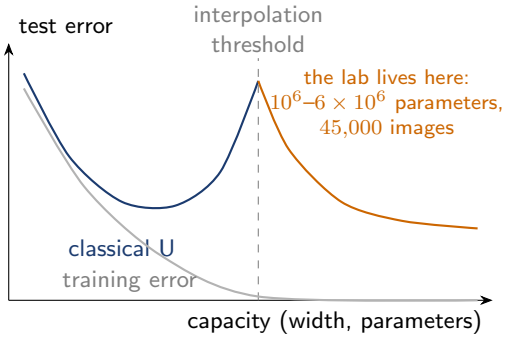
The gap, and why capacity is not the knob



Belkin et al. 2019; Nakkiran et al. 2020, "deep double descent".

- What we want is the loss on new data (the risk); the gradient only sees the training set. The gap is the price of choosing θ on that set: E4, training loss 0.003, validation 1.4 at best.

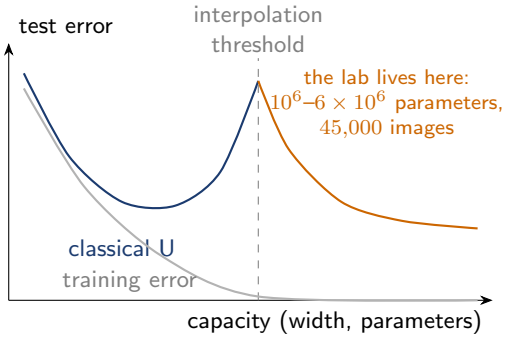
The gap, and why capacity is not the knob



Belkin et al. 2019; Nakkiran et al. 2020, "deep double descent".

- What we want is the loss on new data (the risk); the gradient only sees the training set. The gap is the price of choosing θ on that set: E4, training loss 0.003, validation 1.4 at best.
- Past the point where the network fits everything, the test error *falls again* with capacity ("Double Descent").

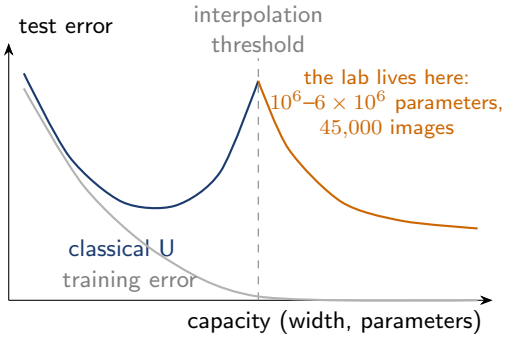
The gap, and why capacity is not the knob



Belkin et al. 2019; Nakkiran et al. 2020, "deep double descent".

- What we want is the loss on new data (the risk); the gradient only sees the training set. The gap is the price of choosing θ on that set: E4, training loss 0.003, validation 1.4 at best.
- Past the point where the network fits everything, the test error *falls again* with capacity ("Double Descent").
- There are many many "perfect" functions on the training data. We want to **bias** the model towards choosing better ones.

The gap, and why capacity is not the knob



Belkin et al. 2019; Nakkiran et al. 2020, "deep double descent".

- What we want is the loss on new data (the risk); the gradient only sees the training set. The gap is the price of choosing θ on that set: E4, training loss 0.003, validation 1.4 at best.
- Past the point where the network fits everything, the test error *falls again* with capacity ("Double Descent").
- There are many many "perfect" functions on the training data. We want to **bias** the model towards choosing better ones.
- The knobs are not parameter counts. They are data & priors (today) and the architecture's inductive bias (next week).

The remedies are priors

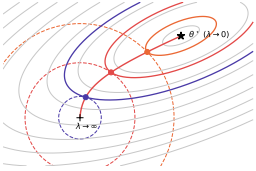
remedy	the prior / the constraint	on this MLP
more data	none; the gap $\to 0$ with n	E6
augmentation	the label is invariant under a group G	today
weight decay	$\theta \sim \mathcal{N}(0, \lambda^{-1} I)$, MAP (session 2)	today
early stopping	short training = small weights (next frame)	E4
dropout	noise on hidden units; an ensemble	today
architecture	the *right* functions	Next week

Weight decay

Weight decay: add a penalty on the size of the weights to the loss,

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2,$$

and the gradient step shrinks them every time: $\theta \leftarrow \theta - \eta\nabla L_\lambda = (1 - \eta\lambda)\,\theta - \eta\nabla L(\theta)$ (Adam: AdamW).



Weight decay

Weight decay: add a penalty on the size of the weights to the loss,

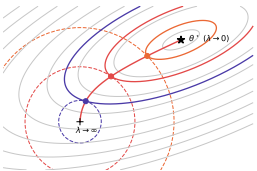
$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2,$$

and the gradient step shrinks them every time: $\theta \leftarrow \theta - \eta\nabla L_\lambda = (1-\eta\lambda)\,\theta - \eta\nabla L(\theta)$ (Adam: AdamW).

On a quadratic, $L = \frac{1}{2}\sum_i \lambda_i(\theta_i - \theta_i^\star)^2$ in the Hessian's eigenbasis, the minimum of L_λ is, mode by mode,

$$\theta_i = \frac{\lambda_i}{\lambda_i + \lambda}\,\theta_i^\star.$$

Sharp modes ($\lambda_i \gg \lambda$) kept: the data pin them. Flat modes ($\lambda_i \ll \lambda$) set to 0: the data barely care.



Weight decay

Weight decay: add a penalty on the size of the weights to the loss,

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2,$$

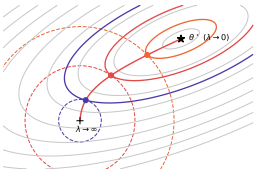
and the gradient step shrinks them every time: $\theta \leftarrow \theta - \eta\nabla L_\lambda = (1-\eta\lambda)\,\theta - \eta\nabla L(\theta)$ (Adam: AdamW).

On a quadratic, $L = \frac{1}{2}\sum_i \lambda_i(\theta_i - \theta_i^\star)^2$ in the Hessian's eigenbasis, the minimum of L_λ is, mode by mode,

$$\theta_i = \frac{\lambda_i}{\lambda_i + \lambda}\,\theta_i^\star.$$

Sharp modes ($\lambda_i \gg \lambda$) kept: the data pin them. Flat modes ($\lambda_i \ll \lambda$) set to 0: the data barely care.

Same thing as a constraint: minimize L on a sphere of fixed radius; λ is the Lagrange multiplier (right: dots where a contour touches the sphere). The radius is not chosen, it is read off after training.



Weight decay

Weight decay: add a penalty on the size of the weights to the loss,

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2,$$

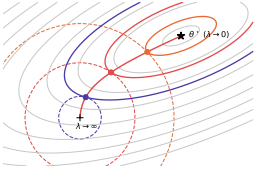
and the gradient step shrinks them every time: $\theta \leftarrow \theta - \eta\nabla L_\lambda = (1-\eta\lambda)\,\theta - \eta\nabla L(\theta)$ (Adam: AdamW).

On a quadratic, $L = \frac{1}{2}\sum_i \lambda_i(\theta_i - \theta_i^\star)^2$ in the Hessian's eigenbasis, the minimum of L_λ is, mode by mode,

$$\theta_i = \frac{\lambda_i}{\lambda_i + \lambda}\,\theta_i^\star.$$

Sharp modes ($\lambda_i \gg \lambda$) kept: the data pin them. Flat modes ($\lambda_i \ll \lambda$) set to 0: the data barely care.

Same thing as a constraint: minimize L on a sphere of fixed radius; λ is the Lagrange multiplier (right: dots where a contour touches the sphere). The radius is not chosen, it is read off after training.



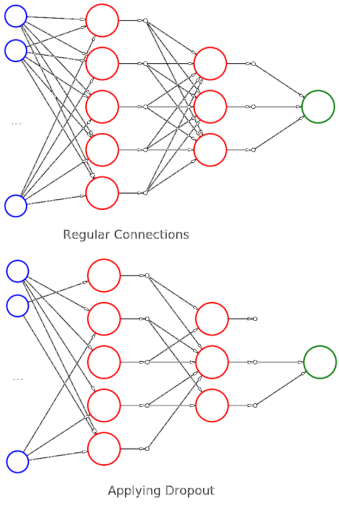
Same thing as a prior: session 2 gave the loss from a likelihood; the penalty is the other half of Bayes, $-\log p(\theta \mid \text{data}) = -\log p(\text{data} \mid \theta) - \log p(\theta) + \text{const}$, with $\theta \sim \mathcal{N}(0, \lambda^{-1} I)$. The minimum of L_λ is the MAP estimate.

Dropout

Training: each hidden unit is zeroed with probability p, independently per example and per step; the survivors are scaled by $1/(1-p)$ so the mean is unchanged:

$$a_j \leftarrow a_j \, \frac{m_j}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p).$$

Evaluation: no mask (`model.eval()`). `nn.Dropout(p)` after the activation; the lab's `dropout` key.



D. V. Godoy, Wikimedia Commons, CC BY 4.0

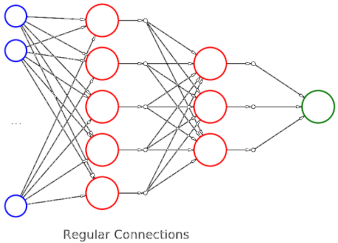
Dropout

Training: each hidden unit is zeroed with probability p, independently per example and per step; the survivors are scaled by $1/(1-p)$ so the mean is unchanged:

$$a_j \leftarrow a_j \, \frac{m_j}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p).$$

Evaluation: no mask (`model.eval()`). `nn.Dropout(p)` after the activation; the lab's `dropout` key.

- noise on the hidden units: no unit can rely on another being there; a different thinned network every step, and at test time roughly their average



D. V. Godoy, Wikimedia Commons, CC BY 4.0

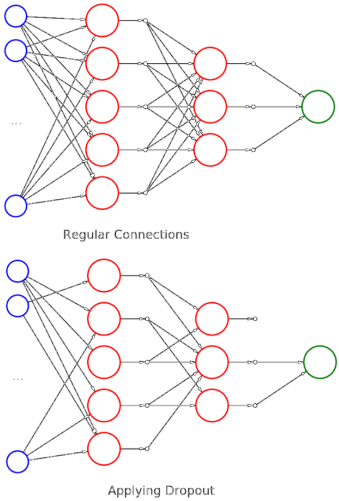
Dropout

Training: each hidden unit is zeroed with probability p, independently per example and per step; the survivors are scaled by $1/(1-p)$ so the mean is unchanged:

$$a_j \leftarrow a_j \, \frac{m_j}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p).$$

Evaluation: no mask (`model.eval()`). `nn.Dropout(p)` after the activation; the lab's `dropout` key.

- noise on the hidden units: no unit can rely on another being there; a different thinned network every step, and at test time roughly their average
- You'll play with this in E10.



D. V. Godoy, Wikimedia Commons, CC BY 4.0

Augmentation

Augmentation: a symmetry, stated as data

original



Augmentation: a symmetry, stated as data

original



horizontal flip



Augmentation: a symmetry, stated as data

original



horizontal flip



scale 1.3



Augmentation: a symmetry, stated as data

original



horizontal flip

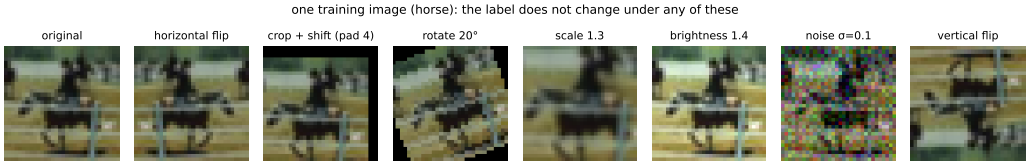


scale 1.3



What if the data has a symmetry G: $p(y \mid g\mathbf{x}) = p(y \mid \mathbf{x})$ for every $g \in G$? How do we use it?

Augmentation: a symmetry, stated as data



What if the data has a symmetry G: $p(y \mid g\mathbf{x}) = p(y \mid \mathbf{x})$ for every $g \in G$? How do we use it?

Idea: a larger training set, $(g\mathbf{x}, y)$ for random $g \in G$, a fresh g every step: **augmentation**. In you experiments today the network will never see the same input twice.

Augmentation for scientific data, and when it is wrong

G is whatever leaves the label alone:

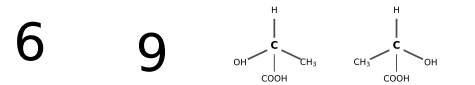
- rotations and flips: microscopy, histology, astronomy, satellite patches (no preferred orientation)
- time shifts, resampling: signals, spectra on a shifted grid
- noise at the instrument's level; gain and offset changes; a missing channel
- permutations: sets of particles, atoms; units

Augmentation for scientific data, and when it is wrong

G is whatever leaves the label alone:

- rotations and flips: microscopy, histology, astronomy, satellite patches (no preferred orientation)
- time shifts, resampling: signals, spectra on a shifted grid
- noise at the instrument's level; gain and offset changes; a missing channel
- permutations: sets of particles, atoms; units

a 6 | the same, rotated 180° | a chiral centre | its mirror image: a different molecule

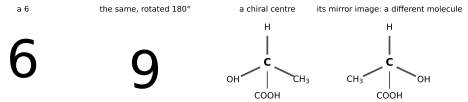


Be careful though

Augmentation for scientific data, and when it is wrong

G is whatever leaves the label alone:

- rotations and flips: microscopy, histology, astronomy, satellite patches (no preferred orientation)
- time shifts, resampling: signals, spectra on a shifted grid
- noise at the instrument's level; gain and offset changes; a missing channel
- permutations: sets of particles, atoms; units



Be careful though

Key point

Know a symmetry? Use it: as augmentation today, as architecture next session. Worth more than any optimizer.

Hands on!

Before the lab: pull the repo

```
cd aims-lab-session2
git pull
```

Since last week:

- `lab_mlp.pdf`: Part C is today's E12–E15, with the knobs from the slides
- `mlp_lab2.py`: `mlp_lab.py` plus `optimizer`, `momentum`, `weight_decay`, `schedule`, `init`, `batchnorm`, `augment`; same defaults, same interface

`mlp_lab.py` and the data are unchanged: nothing to download again.

Now and in HW: E9–E15, to the end of Part C

```
from mlp_lab import train # Part B: E9-E11
from mlp_lab2 import train # Part C: E12-E15
r = train({"optimizer": "adam", "lr": 1e-3,
           "augment": True, "epochs": 1})
r["summary"] # as before
r["history"]["lr"] # new: the learning rate
    per epoch
```

Your E0 pipeline works unchanged after the import.

Sheet: `lab_mlp.pdf`, Parts B and C. Do not edit the scripts.

In groups. Before each run, one line: what do you expect.

9. weight decay: which λ underfits?
10. dropout: where does the validation loss turn now?
11. your best MLP, chosen on validation, then **one** test run
12. the optimizers: who gets where, and how fast
13. depth, fixed: He init and BatchNorm on E7's networks
14. augmentation: does the validation loss still turn?
15. everything together, test once

As before: 10–30 min of compute. Predict first on every run; add the runs to your report issue.

Two failure modes, one table

cause	symptom (lab)	fix
conditioning	crawl / oscillate / diverge with η (E2, E8)	normalize inputs; momentum; Adam
noise	floor, jumping numbers (E1, E5)	schedule to zero; $\eta \propto B$
depth	$\ln 10$ forever (E7)	He init; BatchNorm; skip connections
memorization	training 100%, validation 55 (E4)	early stopping, weight decay, dropout
no invariances	same	augmentation
wrong ansatz	59% ceiling for an MLP on pixels	architecture (next session)

Two failure modes, one table

cause	symptom (lab)	fix
conditioning	crawl / oscillate / diverge with η (E2, E8)	normalize inputs; momentum; Adam
noise	floor, jumping numbers (E1, E5)	schedule to zero; $\eta \propto B$
depth	$\ln 10$ forever (E7)	He init; BatchNorm; skip connections
memorization	training 100%, validation 55 (E4)	early stopping, weight decay, dropout
no invariances	same	augmentation
wrong ansatz	59% ceiling for an MLP on pixels	architecture (next session)

Key point

Training loss $\to 0$: optimization is solved. Priors closed four points of the gap. The MLP does not know which pixels are neighbours; a CNN does. Thursday.